Snapdragon AI Lab

Building and Deploying a TinyML Model with Snapdragon AI PC, Edge Impulse, and Arduino UNO Q

Instructions for Hands-on Workshop • Duration: 4 hours

Welcome!

In this session you will build a machine-learning model that detects a human fall from wrist motion, then deploy it to a real device—the Arduino UNO Q—and watch it react to movement live.

Detecting falls from the wrist is difficult with traditional rule-based methods alone—a simple 'if acceleration is high, it's a fall' threshold is fooled by everyday arm movements. Machine learning changes that: by learning the actual pattern of a fall rather than a single threshold, it makes reliable detection achievable. And by pairing the ML model with a small decision layer (a state machine that confirms a fall before raising an alarm), we can build something genuinely robust. By the end you will have built a working detector, deployed it to hardware, and understood how ML and classical logic complement each other—the core idea behind most real intelligent systems.

What you will accomplish today

- Upload a curated accelerometer dataset (fall and not_fall) into Edge Impulse.
- Design a signal-processing block and extract spectral features from the motion data.
- Train a small neural network and read its confusion matrix critically.
- Deploy the trained model to your group's Arduino UNO Q using Arduino App Lab.
- Validate it live: trigger a fall and watch the prediction respond in a browser dashboard.

How today works

Setting	Detail
Format	The instructor builds each stage live; you follow along on your own Edge Impulse project.
Hardware	One Arduino UNO Q hardware board per group—every group deploys and validates their own board.
Time	4 hours

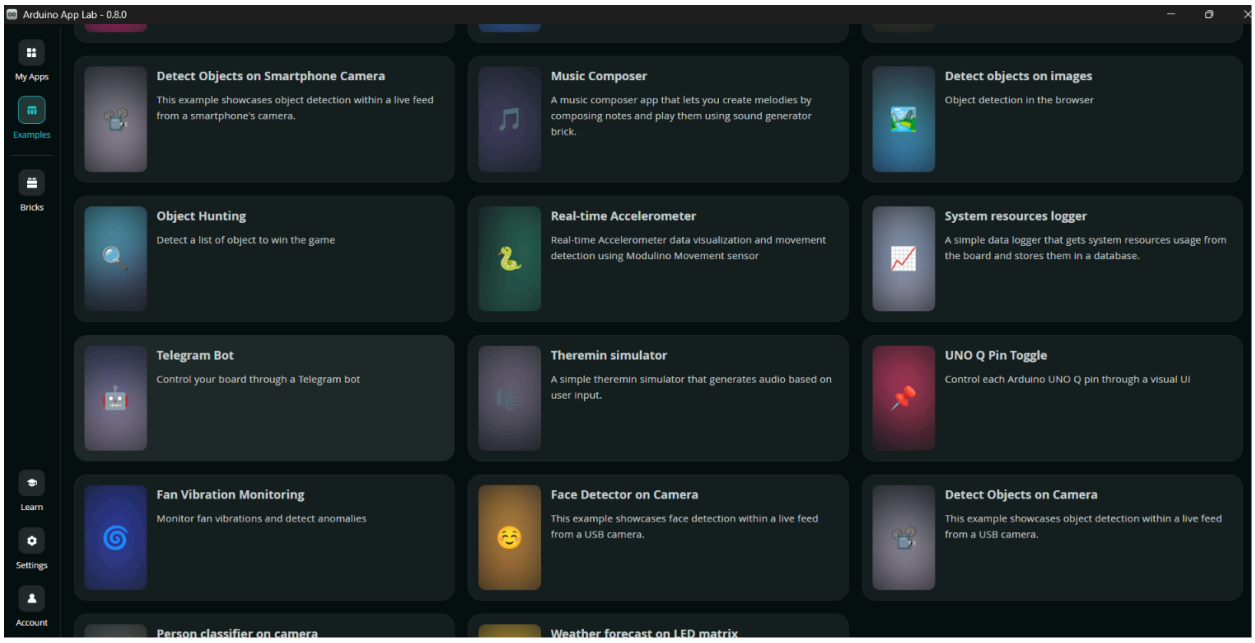
Setting Up the Hardware

Before diving into the data prep and machine learning mechanics, let's connect your hardware and see how the Arduino UNO Q captures and streams physical movement in real time. This quick test ensures your environment is fully set up and gives you a visual baseline for raw accelerometer data.

Hands-On Prelude: Sense Motion

Step 1 — Open the Examples Menu

Inside the Arduino App Lab interface, navigate to Examples > Real-Time Accelerometer.



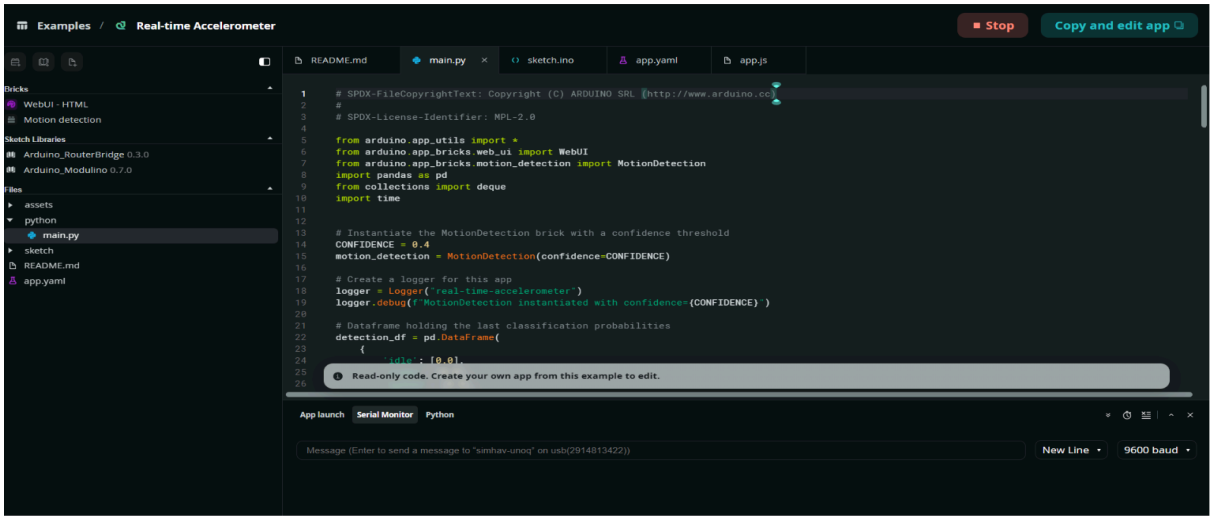
Step 2 — Run the App

Click the RUN icon in the top toolbar to build and send this test program to your board.

Step 3 — Confirm Execution

Watch the terminal status until it displays:

“Real-time accelerometer is now running."



What is happening: *The system automatically launches a real-time web dashboard. Pick up your Arduino board and move it around. The dashboard continuously plots your physical movements as live accelerometer waveforms and outputs active movement tracking percentages.*

Build and Train the fall detection model

Background: The Data and the Pre-Work

The dataset for this activity is WEDA-FALL (Wrist Elderly Daily Activity and Fall dataset), recorded at 50 Hz from a smartwatch worn on the wrist. It is special because it includes real elderly participants and because its everyday activities were deliberately chosen to resemble falls—making it an honest, challenging dataset rather than an artificially easy one.

Pre-processing done before the workshop (for your understanding — you will NOT run this today)

Raw wearable data is messy. Before today, the data was cleaned with a Python script so we could focus on the machine-learning workflow. The script performed four steps that are worth knowing about:

- Resampling to uniform 50 Hz. The smartwatch delivers samples in irregular bursts. Frequency analysis (the FFT) is only valid on evenly spaced samples, so each recording was interpolated onto a clean 20 ms grid.
- Trimming falls to the real event. Each fall recording is ~10 seconds, but the actual fall lasts ~3 seconds. Using the dataset's official labeled timestamps, each fall was cropped to its true event so the model learns falling—not the walking before it.
- Segmenting daily activities. Activity recordings are long and uniform, so they were cut into matching 3-second segments.
- Balancing the classes. The number of activity segments per recording was capped so that 'fall' and 'not_fall' examples are not wildly imbalanced.

The result is two clean folders — fall and not_fall — with every sample exactly 3 seconds long. These are what you will upload.

Why this matters

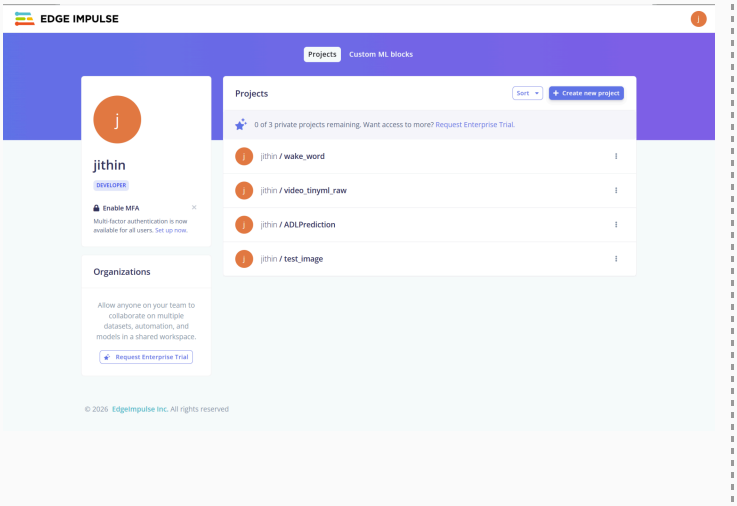
Roughly 80% of real machine-learning work is data preparation, not modeling. We did it for you today to fit the time, but the lesson stands: a model is only as good as the data it learns from. Clean, well-labelled, balanced data is what makes the next two hours work.

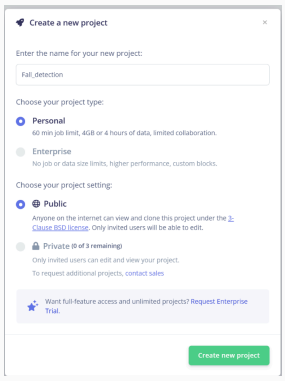
Part 1 — Data and Feature Extraction

Goal: get the motion data into Edge Impulse and turn raw accelerometer signals into numerical features a model can learn from.

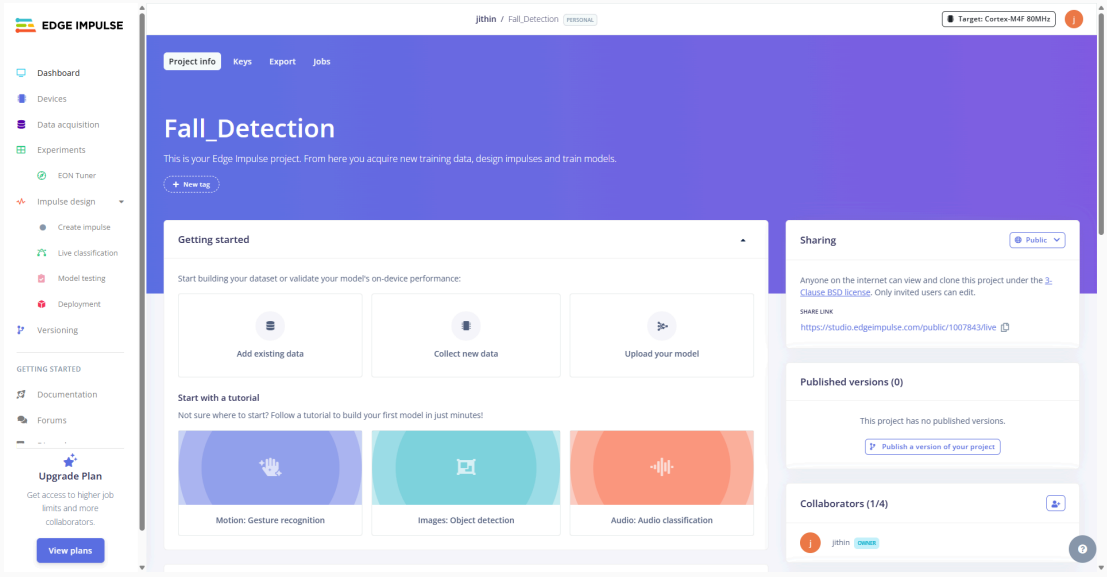
Step 1.1 — Create your Edge Impulse project

1. Sign in at edgeimpulse.com and create a new project by clicking the top right button “Create New Project." Name it something like fall-detection-yourname.



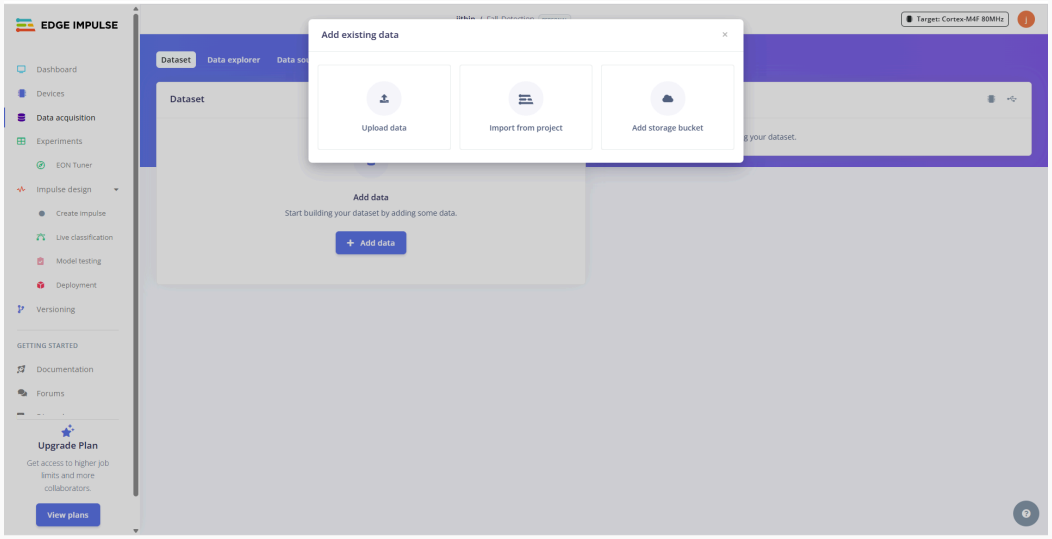


2. Once created, you will be redirected to the project dashboard that might look like this.

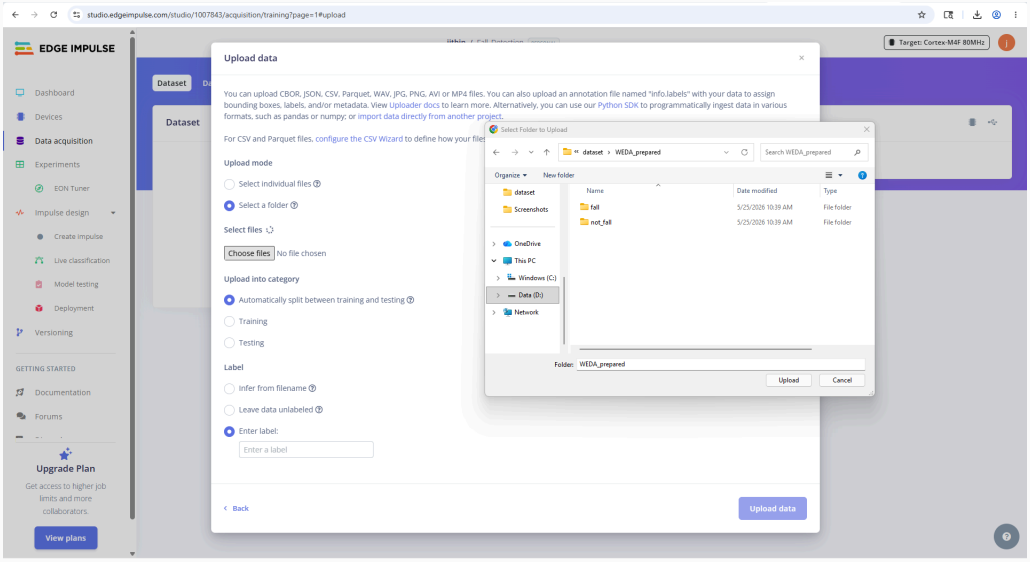


Step 1.2 — Upload the dataset

1. Open the Data acquisition tab from the side panel, then click the “Add data” button, which might give you a pop-up with 3 options. Choose the “Upload data” option among them to upload from your local machine.



2. Select the fall folder of prepared CSV files. Set the label to fall.

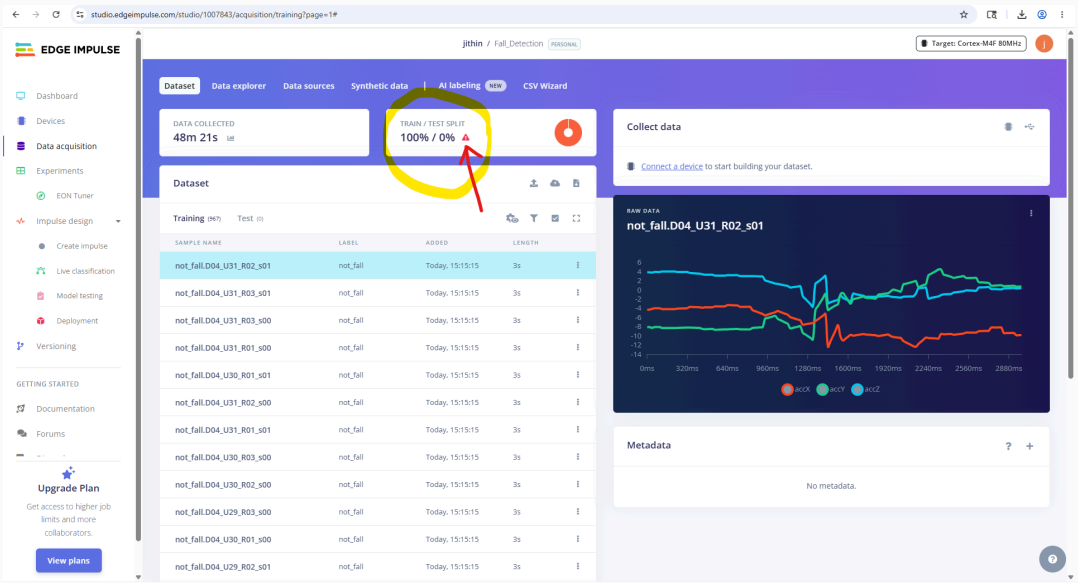


3. Repeat for the not_fall folder, labeled not_fall.

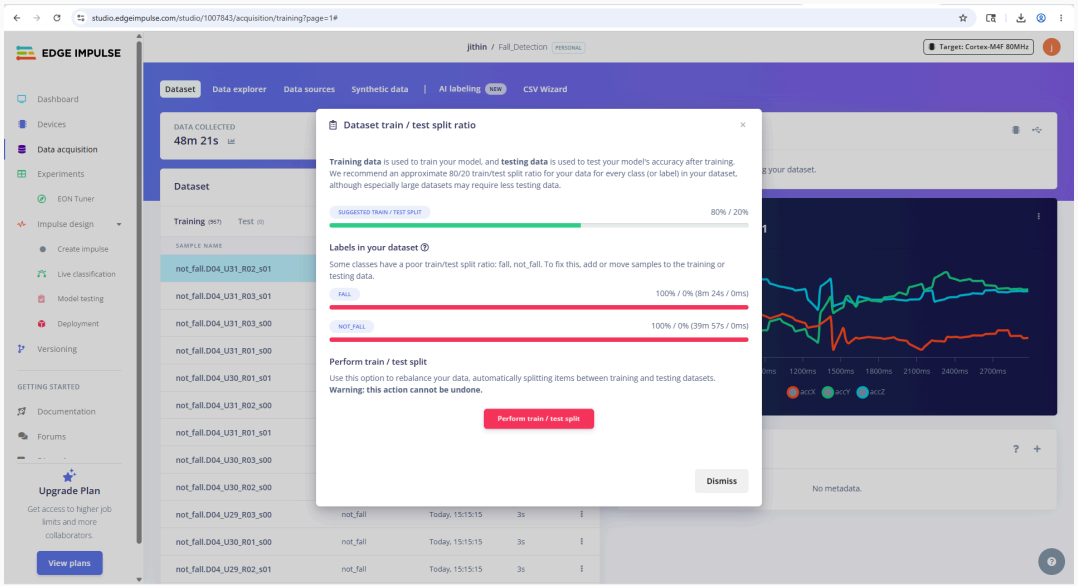
Step 1.3—Split train/test by SUBJECT

This is the single **most important data step**. We must hold out some people entirely for testing.

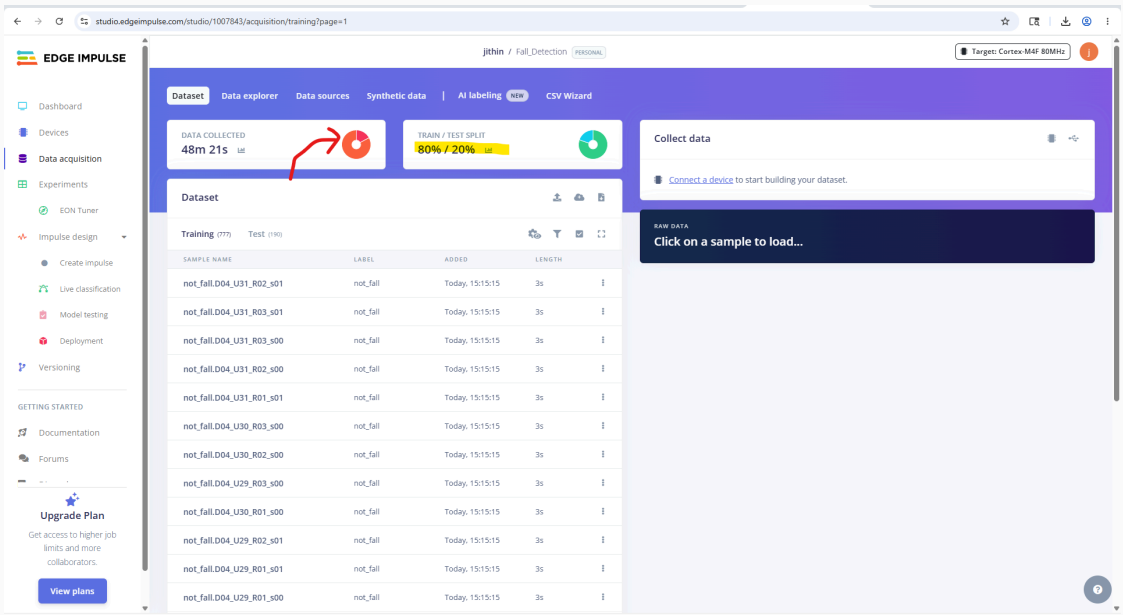
1. Click on the warning triangle icon with a "Train/Test Split" label



2. Click on the "perform split" button to split the data into 80/20 train/split ratio

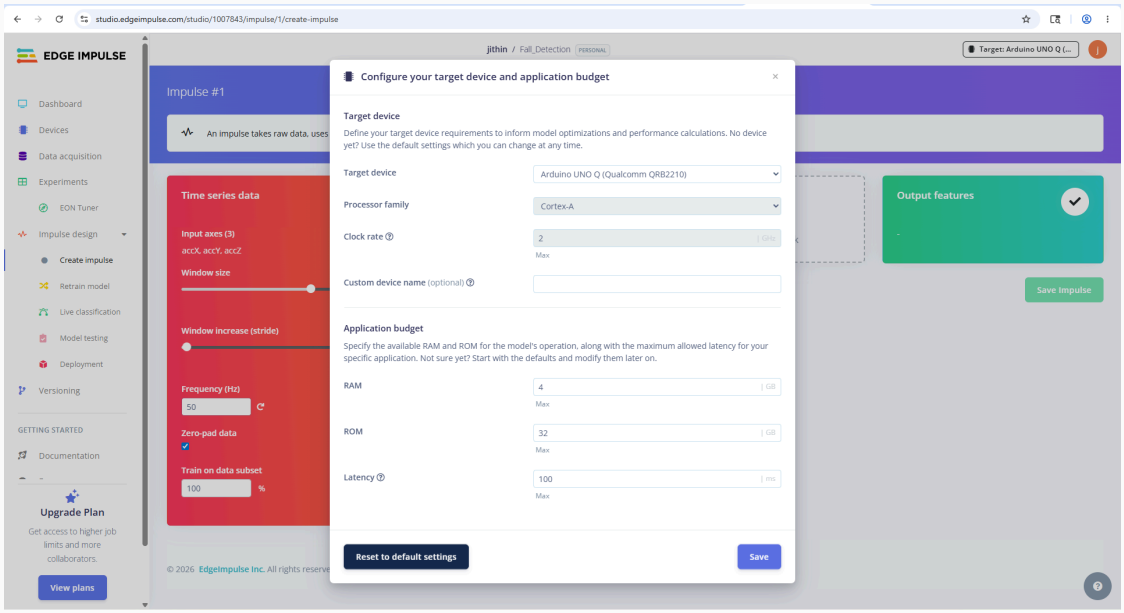


You might observe that the data is imbalanced here; the amount of fall data is less than the not_fall data that we uploaded. We will be taking some actions to avoid certain consequences that might occur because of this later.

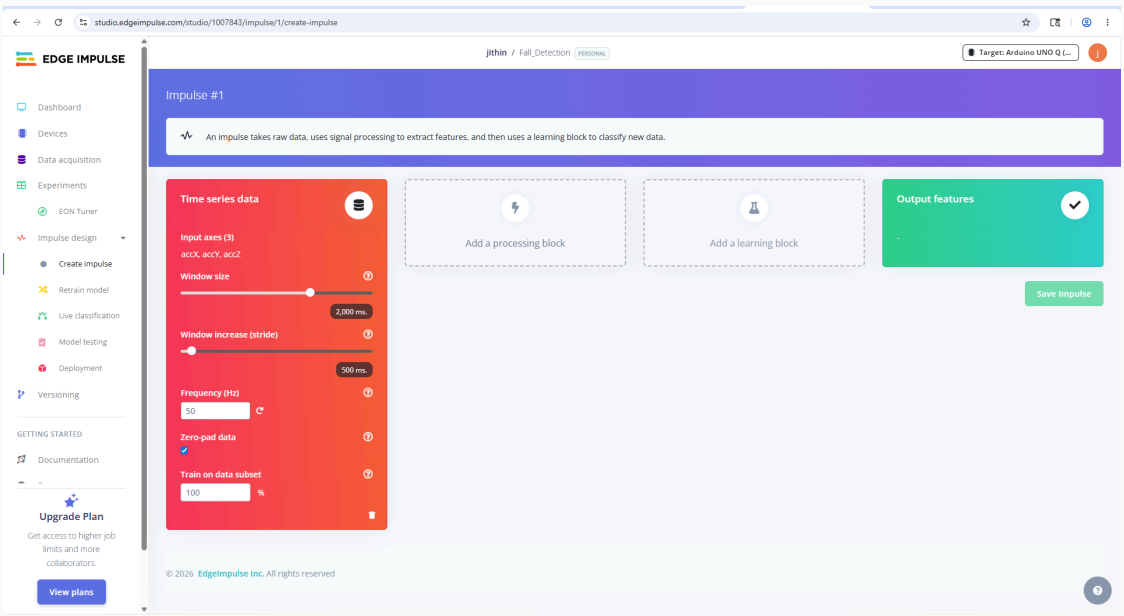


Step 1.4 — Create the Impulse

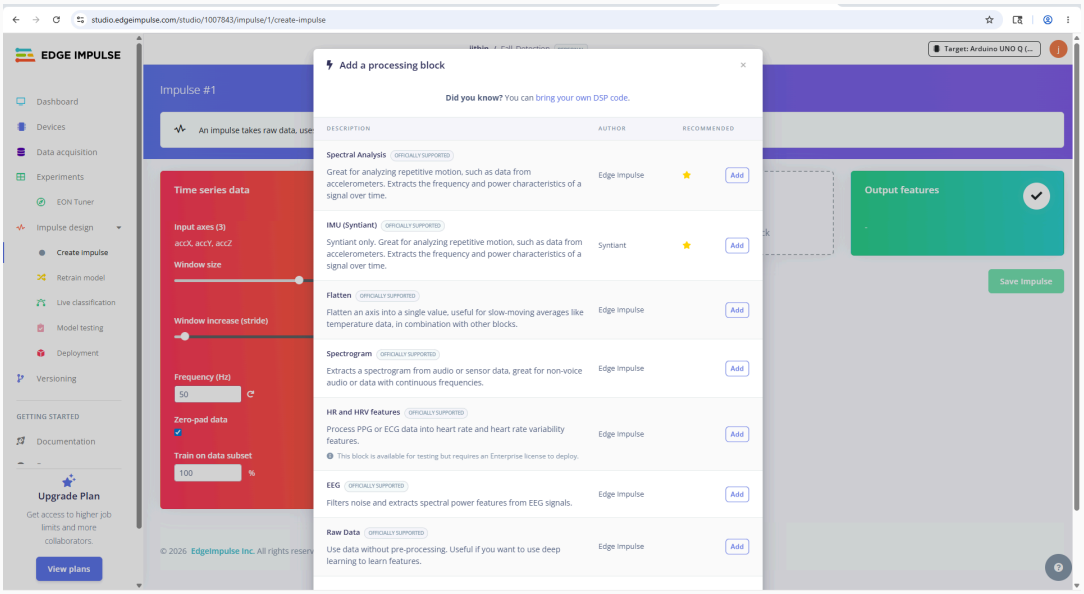
3. Go to Create Impulse under the Impulse design tab from the side panel. When you click this option, usually it gives you a pop-up to choose the “Target device and Application budget.”



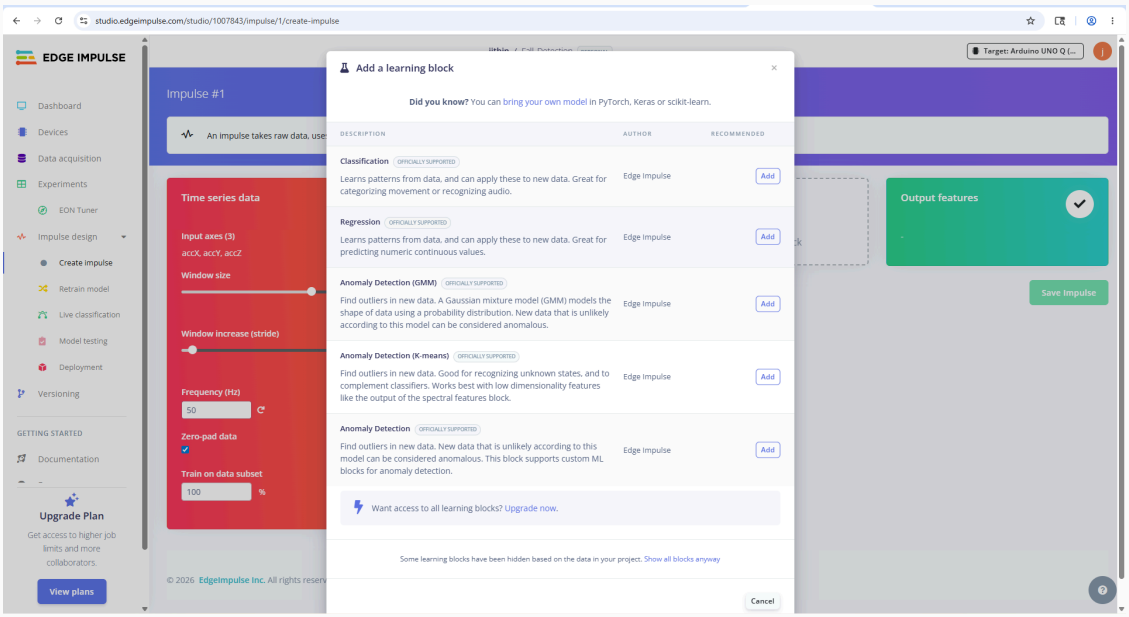
4. Set the time-series window: **Window size = 2000 ms, Window increase (stride) = 500 ms.**



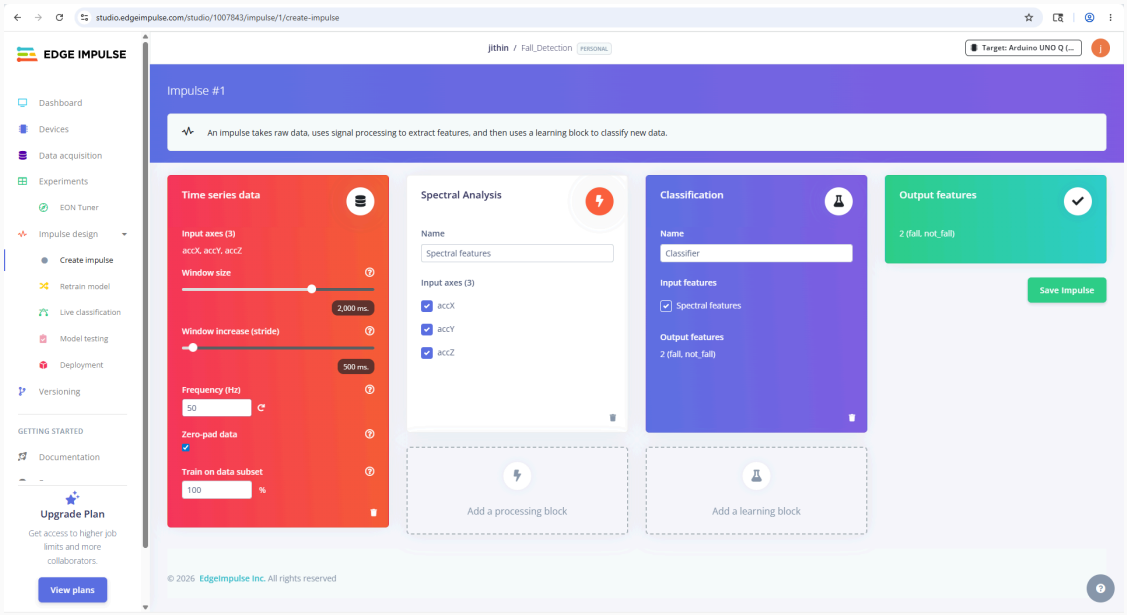
5. Add a **Spectral Analysis** processing block by clicking onto “Add a processing block." You will get a pop-up with multiple options; choose **Spectral Analysis**.



6. Add a **Classification** (Neural Network) learning block. Save the impulse



7. Now we have the complete Impulse design ready. Time series data input with a signal processing block to extract features and then a **classification** learning block to classify the data. As you can see, the output feature block automatically identifies the output classes.

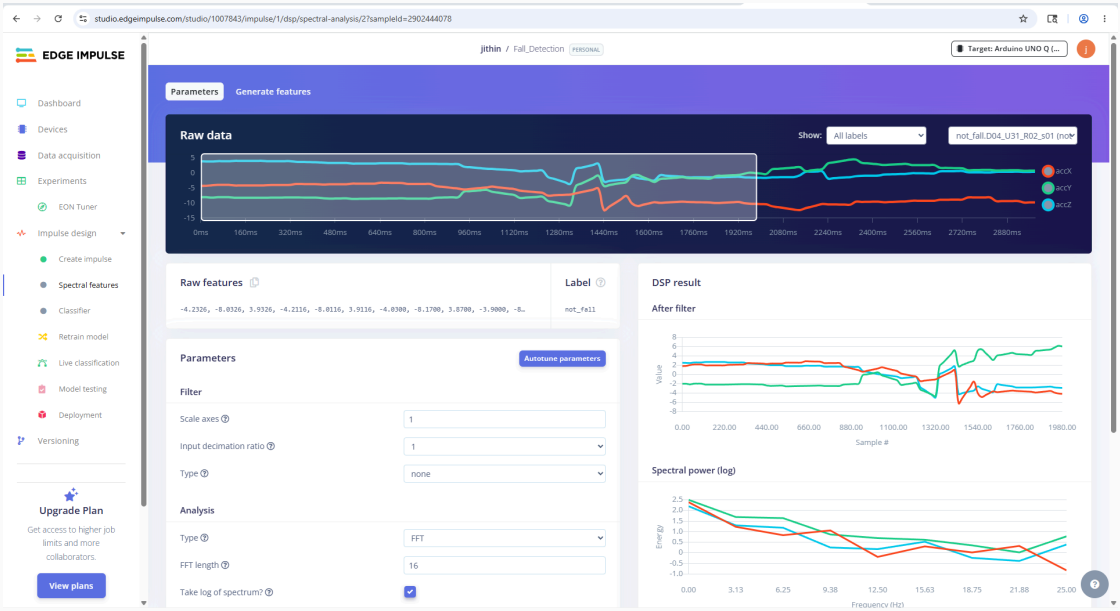


💡 **What's happening:** *Each data sample is 3 seconds long, but the window is 2 seconds, so the window slides across each file, producing ~3 overlapping windows per sample. This multiplies our training examples and teaches the model to recognise a fall no matter where in the window it occurs.*

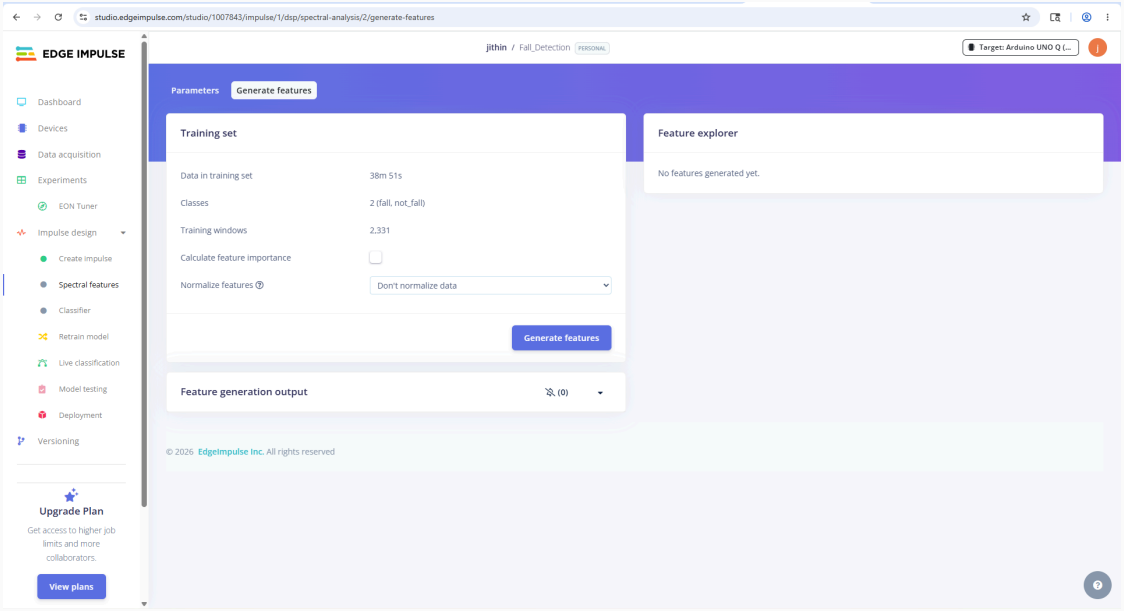
Step 1.5 — Configure spectral features

1. Open the Spectral features block and set the following:

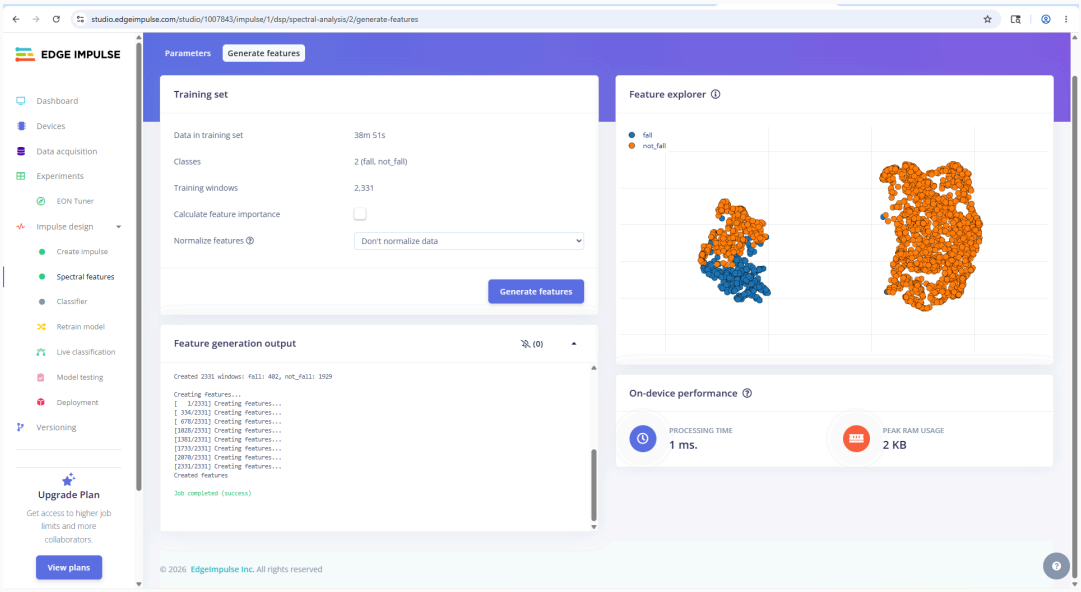
Parameter	Value
Filter type	No filter
FFT length	16



2. Click Save parameters, then Generate features.



3. The feature explorer block visualises a 2D/3D diagram based on the parameters and the input data.



✓ **Expected:** *Feature generation completes, and the feature explorer shows fall (one colour) and not_fall (another) as point clouds.*

Reading the Feature Explorer honestly

The explorer shows your features squashed from many dimensions down to 2D or 3D so a human can look at them. Two points that overlap in this flattened picture may be perfectly separable in the full feature space the model actually uses. So:

- Clean separation here is a good sign—but it is a preview, not the verdict.
- Some overlap is expected and honest — a few activities genuinely resemble falls at the wrist.
- The real verdict comes from the confusion matrix on test data.

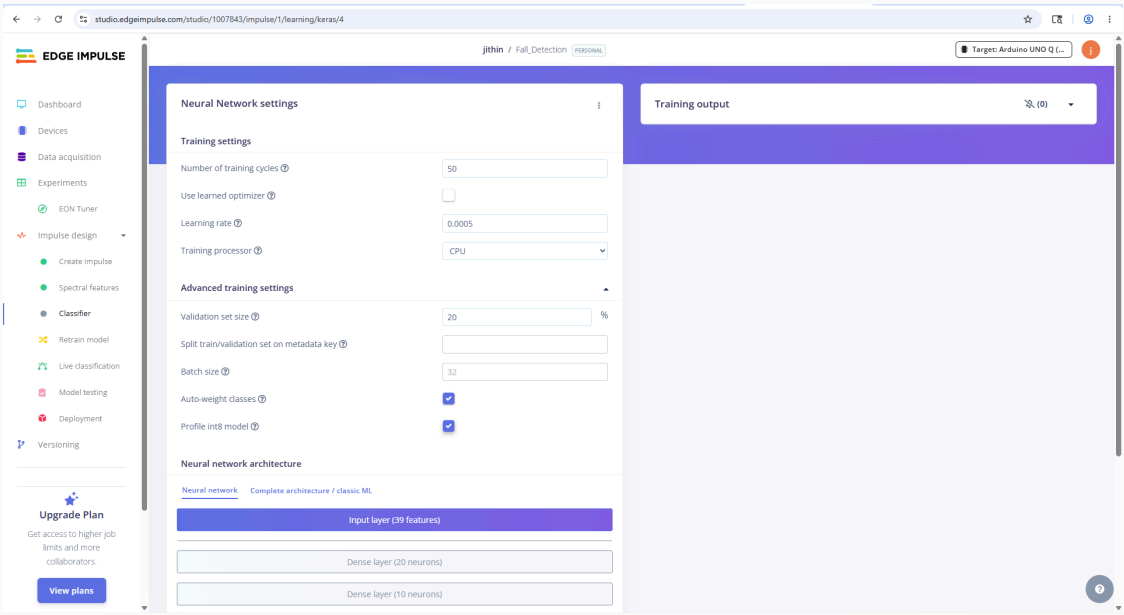
Part 2 — Train and Test the Model

Goal: train a neural network on the features and judge it properly using the confusion matrix.

Step 2.1 — Configure the neural network

1. Open the Classifier tab and set **Training cycles = 50**, **Learning rate = 0.0005** (defaults are usually fine).

2. In Advanced settings, enable **Auto-weight classes** (compensates for the class imbalance). Leave the default small architecture (two dense layers, e.g. 20 and 10 neurons).



💡 **What's happening:** *Auto-weight tells the model to pay proportionally more attention to the rarer 'fall' class, so it is not overwhelmed by the more numerous 'not_fall' examples.*

Step 2.2 — Train

1. Click Save & train. Training takes a couple of minutes. Watch the training output and the accuracy/loss as they update.

✓ **Expected:** *Training completes and a confusion matrix appears for the validation set.*

Step 2.3 — Read the confusion matrix critically

Do not just read the headline accuracy. For a fall detector, focus on the fall row.

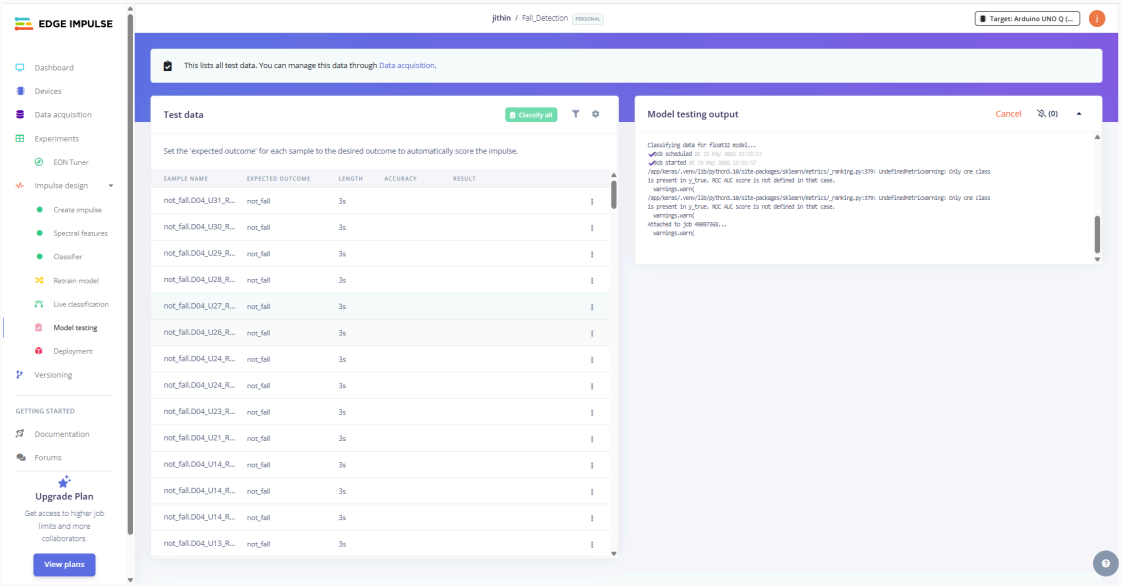
Metric	What it tells you
Accuracy (overall)	Can be misleading — the larger not_fall class inflates it.
Fall recall	Of all real falls, how many were caught? Missing a fall is the worst error. Watch this number most.
Fall precision	When the model says 'fall,' how often is it right? Low precision = false alarms.
Fall F1	A single balance of recall and precision for the fall class.

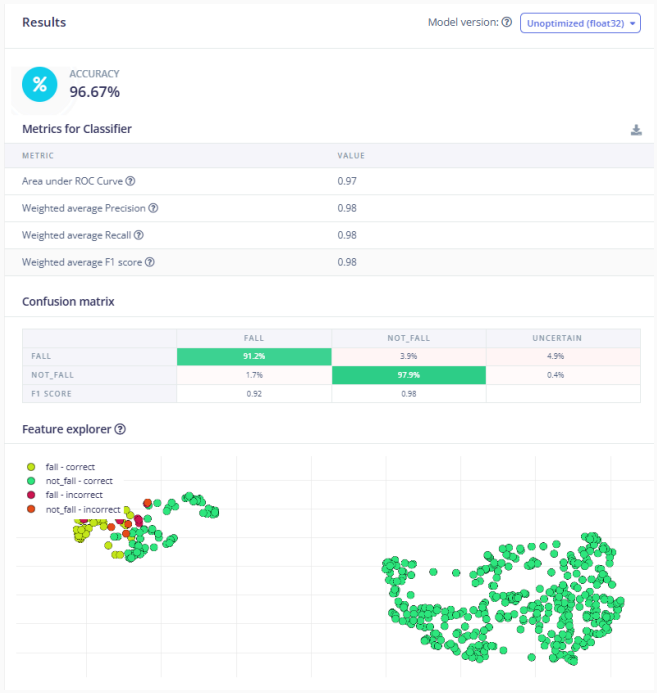
The question that matters

If your watch failed to detect a real fall, an injured person gets no help. If it raises a false alarm, someone is mildly annoyed. So for fall detection we prioritise high recall (catch every fall) and accept some false alarms. Note which kinds of activities get misclassified as falls — they are usually the ones that look like falls at the wrist (a stumble, a sudden sit).

Step 2.4 — Test on held-out data

1. Open Model testing and click Classify all.
2. This will feed the Test data set that model has never seen before.





✓ **Expected:** *You get a test-set accuracy and confusion matrix. It is normal for this to be a little lower than the validation numbers — this is the honest, real-world estimate.*

💡 **What's happening:** *Validation data influenced training indirectly; the test set did not. Test performance is the number you would quote to someone asking 'does it actually work?'*

Deploy the Model to the Arduino UNO Q

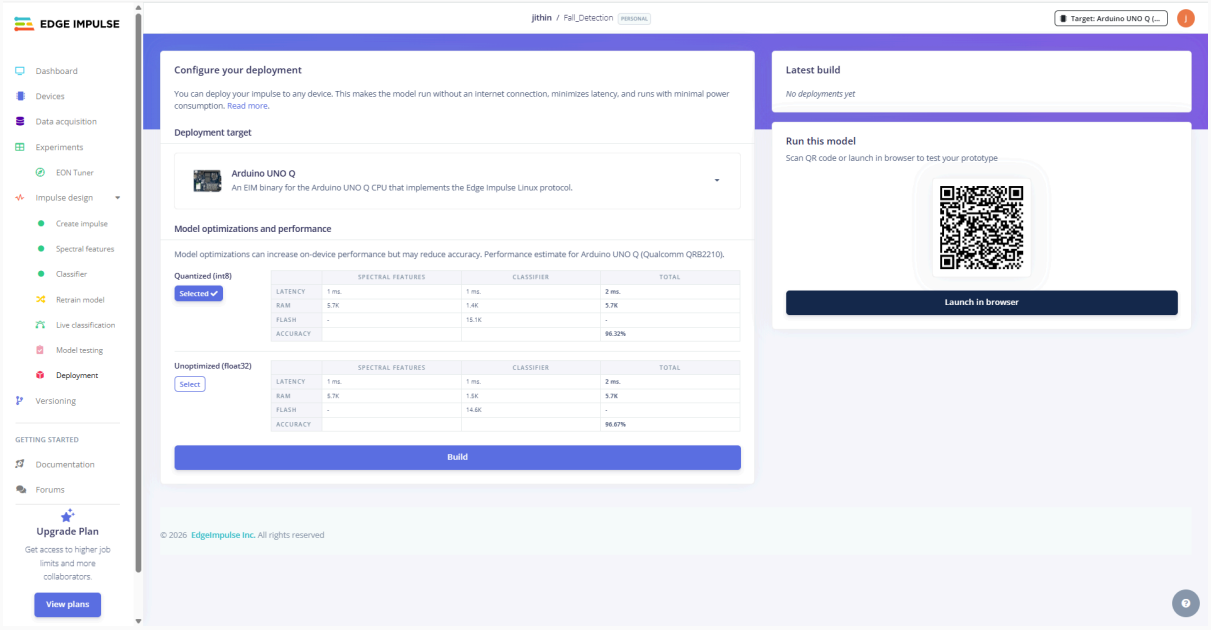
Part 1 — Build and Copy the Model to Arduino UNO Q

Goal: put the trained model onto real hardware and watch it react to motion in real time.

> **About the Arduino UNO Q**
> The UNO Q is a dual-brain board: a Linux-capable Qualcomm processor plus a microcontroller. A small accelerometer module (Modulino Movement) supplies live motion. The microcontroller reads the sensor and passes samples to the Linux side, where your model runs and a browser dashboard shows the result.

Step 1.1—Build the model for the UNO Q

1. In Edge Impulse, open the Deployment tab.
2. Select the deployment option for **Arduino UNO Q** (build as a library / App Lab model).
3. Click Build. Edge Impulse compiles the model and downloads a .eim file to your computer.



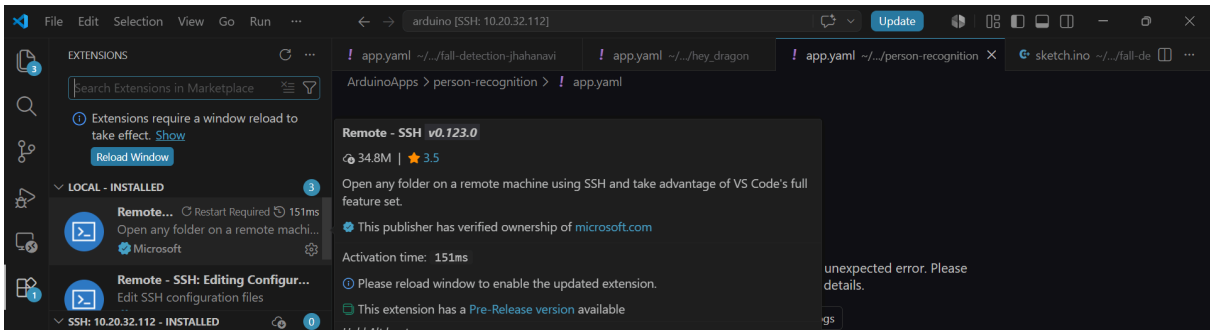
✓ **Expected:** *A model package builds successfully and is downloaded, ready to import into Arduino App Lab.*

Step 1.2 — Setting up VS Code and Connecting via Wireless SSH

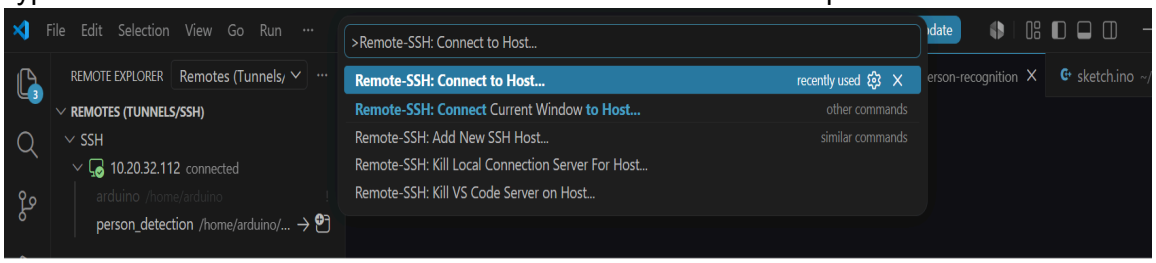
Now that the board is broadcasting on your wireless network, you can completely close the App Lab and move to VS Code.

1. Install the Remote-SSH Extension (skip if already installed)

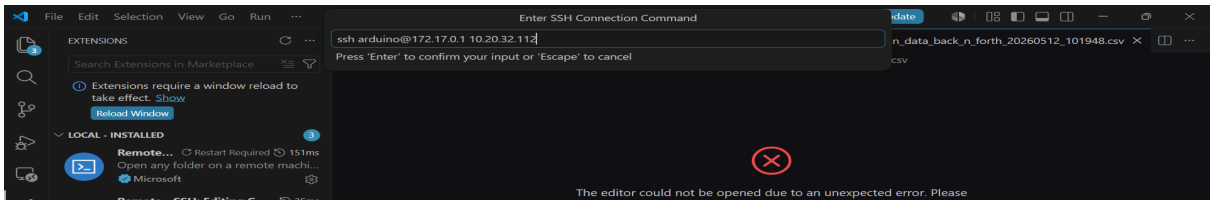
2. Open VS Code and open the Extensions view by clicking the square icon on the left sidebar (or press Ctrl+Shift+X).
3. Search for Remote - SSH (published by Microsoft) and click Install.



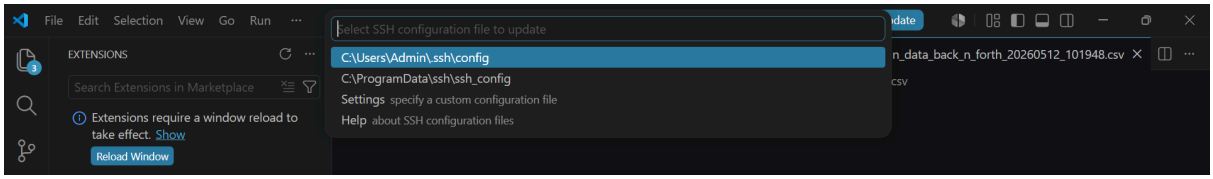
4. Press Ctrl+Shift+P (Windows/Linux) or Cmd+Shift+P (Mac) to open the VS Code Command Palette.
5. Type Remote-SSH: Connect to Host... and select it from the dropdown menu.



6. Click Add New SSH Host... and type the connection string using your board's unique Wi-Fi IP address:



7. (Example: ssh arduino@192.168.1.45)
8. Select your preferred local configuration file directory (the first user directory path option) to save this profile entry.

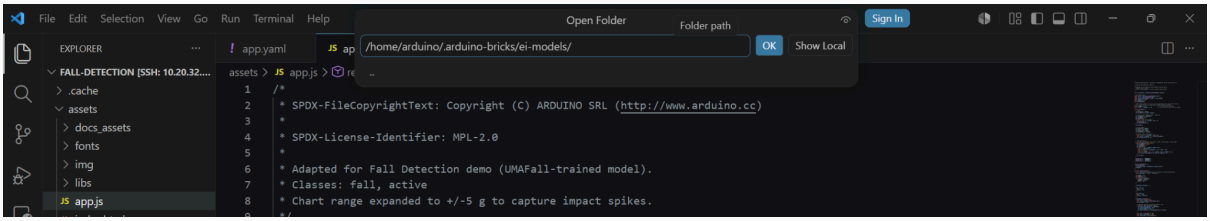


9. A notification popup will appear in the bottom-right corner. Click Connect to launch a new, dedicated VS Code window.
10. When prompted, select Linux as the target system platform and enter the system account password you created during the initial setup.

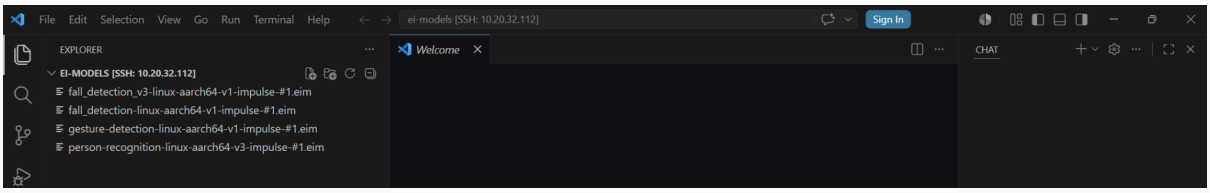
Step 1.2 — Copy the .eim onto the board (VS Code over SSH)

We transfer the model to the UNO Q's filesystem using VS Code's SSH connection.

1. In VS Code, connect to the UNO Q over SSH (Remote-SSH) so you can see the board's filesystem.
2. Navigate to the model directory: **`arduino-bricks/ei-models`**.



3. Copy your downloaded `.eim` file into `arduino-bricks/ei-models` (drag-and-drop or paste in the VS Code explorer).



✓ **Expected:** *Your .eim file is now listed inside arduino-bricks/ei-models on the UNO Q.*

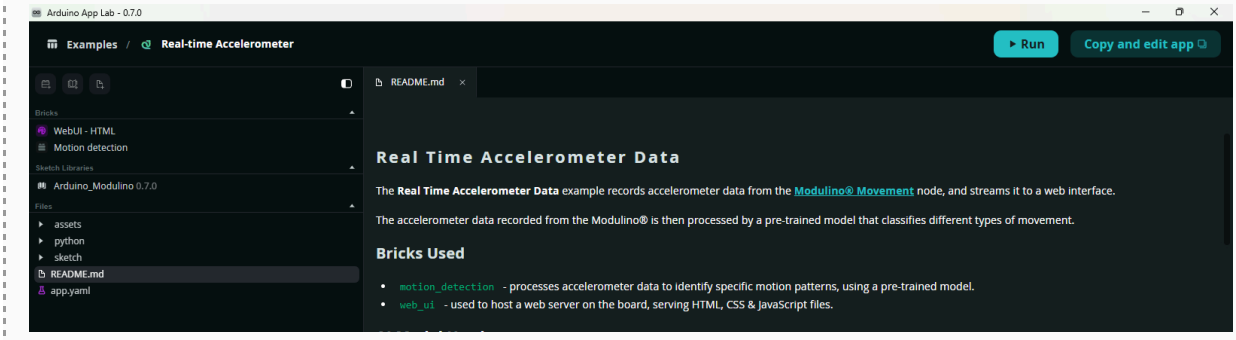
Why place it here?

The on-board application looks in arduino-bricks/ei-models for its model. Putting the .eim there is what tells the app which brain to use. Knowing where a model lives on a device's filesystem — and being able to swap it over SSH — is a normal part of working with embedded Linux systems.

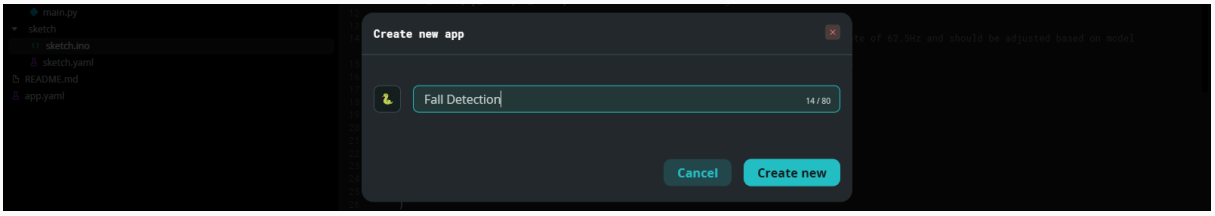
Step 1.3 — Create your own copy of the app

You will customize a copy of Arduino's example app, leaving the original intact.

1. In the Arduino App Lab, open the *Real-Time Accelerometer* example app and click on the top-right button to copy and edit the app.



2. Rename your copy to **Fall Detection**.

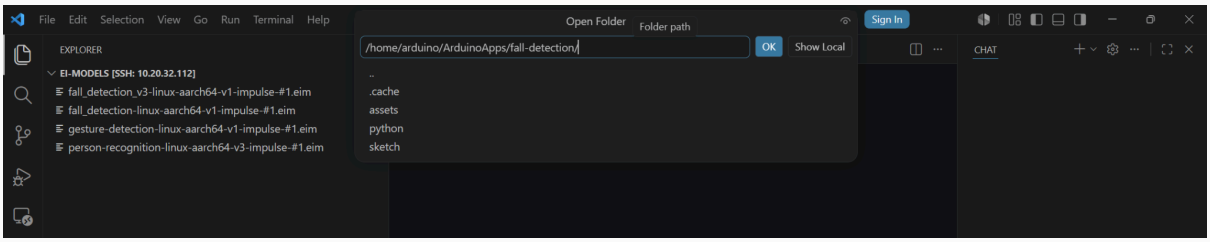


✓ **Expected:** *Your own Fall Detection app appears under My Apps in the App Lab, separate from the original example.*

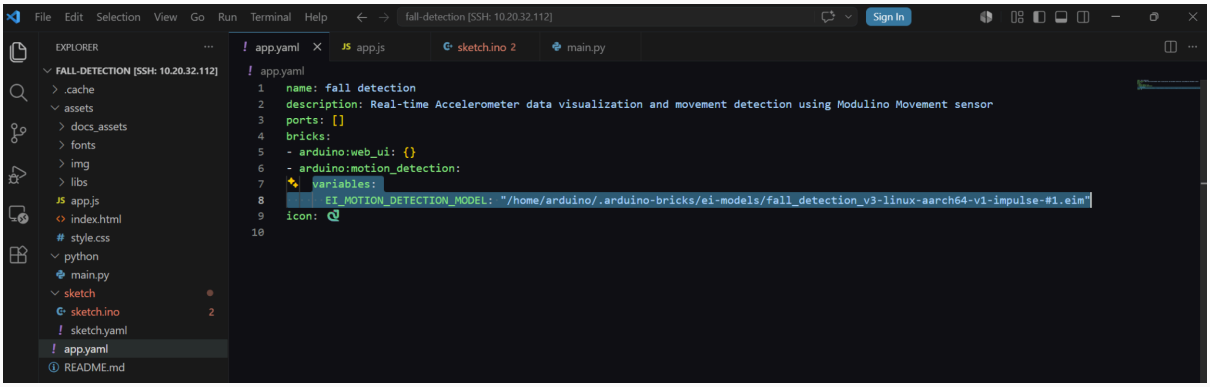
Step 1.4—Point the app at your model (edit app.yaml over SSH)

> **Important: app. yaml must be edited over SSH**
>
> The app.yaml file is NOT editable from inside the Arduino App Lab. You must open it through the VS Code SSH connection to the board. The other files (sketch.ino, main.py, app.js) can be edited in either App Lab or VS Code.

1. In the VS Code SSH window, open the **`ArduinoApps`** folder and find your *Fall Detection* app.



2. Open **`app.yaml`** and set the motion-detection model variable to point at the .eim you copied in Step 3.2.
3. Save the file.



✓ **Expected:** *app.yaml points at your .eim file path under .arduino-bricks/ei-models.*

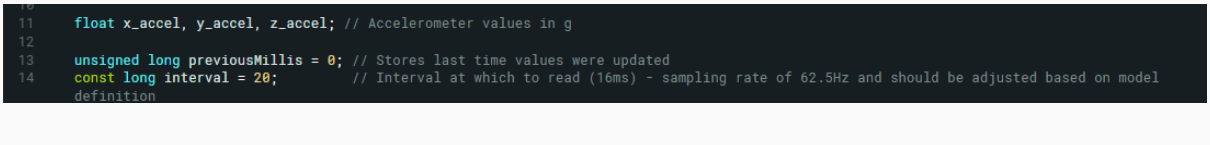
💡 **What's happening:** *This line tells the motion_detection brick which model to load. Without it, the app would not know about the model you copied onto the board. The path must match your .eim filename exactly—a typo here means the app cannot find the model.*

Step 1.5 — Match the sensor feed to the training data (input pipeline)

The model was trained on very specific data. The live sensor feed must match it, or even a perfect model will misfire. These two files, **sketch.ino** and **main.py** need to be edited in App Lab or in VS Code to match the live sensor feed to the model. The full set of conditions the live feed must satisfy:

Setting	Must be	Where set
Sample rate	50 Hz (interval = 20 ms)	sketch.ino
Units	m/s² (multiply g-values by 9.81; gravity ≈ 9.8)	main.py
Range	Wide enough that fall impacts are not clipped	sensor config
Axis orientation	Board still: one axis ≈ 9.8, others ≈ 0	verify (Step 3.6)

1. Let us open **`sketch.ino`** in the Arduino App Lab—set the sampling interval to **20 ms (50 Hz)** to match the training rate.



2. `Similarly open` **`main.py`** in the App Lab—two edits here. Second, make the detection dataframe and the movement-detection callbacks use the model's class names **`fall`** and **`not_fall`** (the example app ships with different class names, e.g. “active”, which must be replaced).

```
def record_sensor_movement(x: float, y: float, z: float):
    logger.debug(f"record_sensor_movement called with raw g-values: x={x}, y={y}, z={z}")
    try:
        # Convert g -> m/s^2 for the detector
        x_ms2 = x * 9.81
        y_ms2 = y * 9.81
        z_ms2 = z * 9.81

        # Forward samples to the motion_detection brick
        motion_detection.accumulate_samples((x_ms2, y_ms2, z_ms2))
        logger.info("Forwarded sensor sample to motion_detection.accumulate_samples")

        # Use raw x,y,z for the lightweight chart
        sample = {
            "t": time.time(),
            "x": float(x_ms2),
            "y": float(y_ms2),
            "z": float(z_ms2)
        }

        # push into local buffer and broadcast to connected UIs
```

```
# Dataframe holding the last classification probabilities
detection_df = pd.DataFrame(
    {
        'fall': [0.0],
        'not_fall':   [0.0]
    }
)
```

```
    try:
        global detection_df
        detection_df = pd.DataFrame(
    {
        'not_fall': [classification.get('not_fall', 0.0)],
        'fall':   [classification.get('fall', 0.0)]
    }
)
```

Note: *If the dataframe and callbacks still use the example's old class names, the model's 'not_fall' prediction has nowhere to go—the value stays at zero, and the dashboard looks broken even though the model is working. The names in code must match the model's class names exactly.*

```
# Register movement callbacks
motion_detection.on_movement_detection('not_fall', on_movement_detected)
motion_detection.on_movement_detection('fall',   on_movement_detected)
logger.info("Registered movement detection callbacks for active,fall")

# Bridge handler: called from the sketch via Bridge.notify("record_sensor_movement", x, y, z)
```

💡 **What's happening:** *If the live data is in the wrong units, the wrong sample rate, or the wrong axis orientation, the numbers the model sees no longer resemble what it learned. Matching the sensor to the training conditions is as important as the model itself.*

Step 1.6 — Match the dashboard labels and Range (output display)

1. Open **`app.js`** (inside the app's `assets` folder) from App Lab—update the displayed class names so the dashboard shows the correct labels and bars. The example app ships with its own class names, so replace them in three places — **`orderedKeys`**, **`icons`**, and **`names`**:

```
function renderClasses(d) {
  // Fall first so the alarming class is visually on top during the demo.
  const orderedKeys = ['fall', 'not_fall'];

  let maxKey = null;
  let maxValue = -1;
  for (const key in d) {
    if (d[key] > maxValue) {
      maxValue = d[key];
      maxKey = key;
    }
  }

  classesRoot.innerHTML = '';

  const icons = {
    fall:   '🤕',
    not_fall: '🏃'
  };

  const names = {
    fall:   'fall',
    not_fall: 'not_fall'
  };
```

✓ **Expected:** *The web dashboard will display 'Fall' and 'Not Fall' (not the example app's original labels).*

2. Also change the Y_RANGE variable in the **app.js** to match the range in m/s2 of the activities

```
// Chart Y-axis range: +/-40 m/s2; falls impacts can hit 40+
// so we widen the range to keep impact visible without clipping).
const Y_RANGE =   40;                // +/- this value on the chart
const Y_STEP  = Y_RANGE * 2 / 8;      // 8 grid divisions across the full range
```

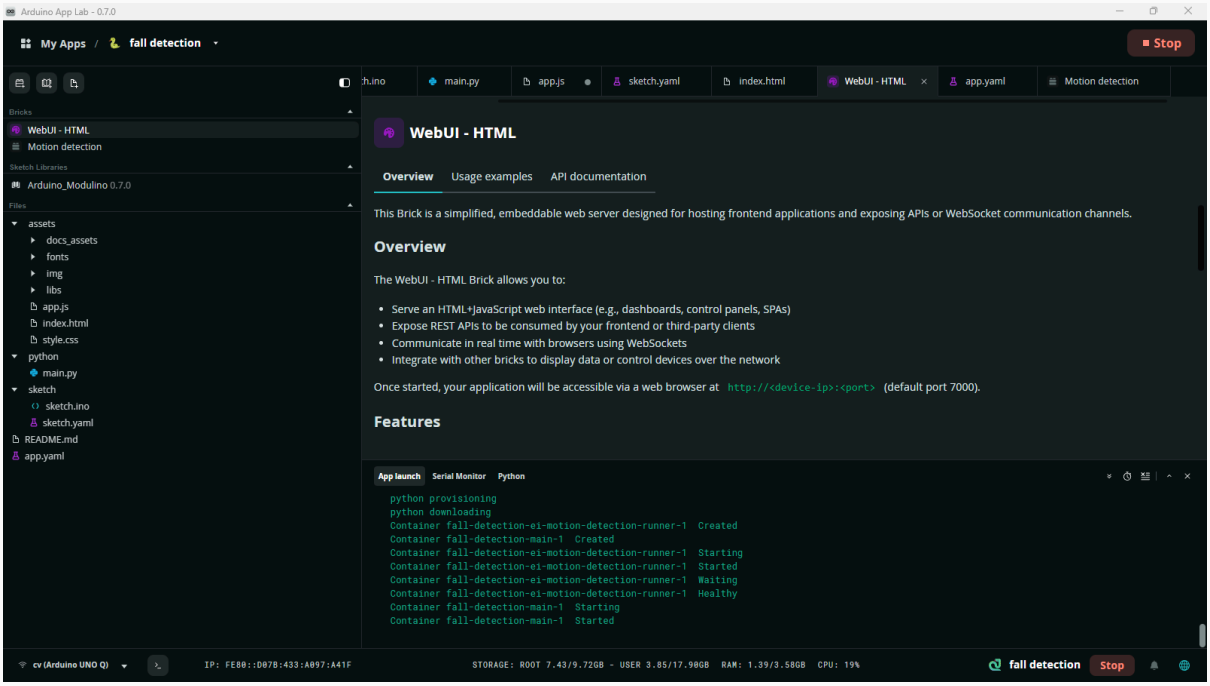
💡 **What's happening:** *The model's edits (app.yaml, sketch.ino, main.py) control what the model sees and predicts; app.js only controls how the result is shown. If the displayed labels do not match the model's class names, the prediction can be correct while the dashboard shows the wrong row—so this edit keeps the display honest.*

Part 2 — Run and Validate Live

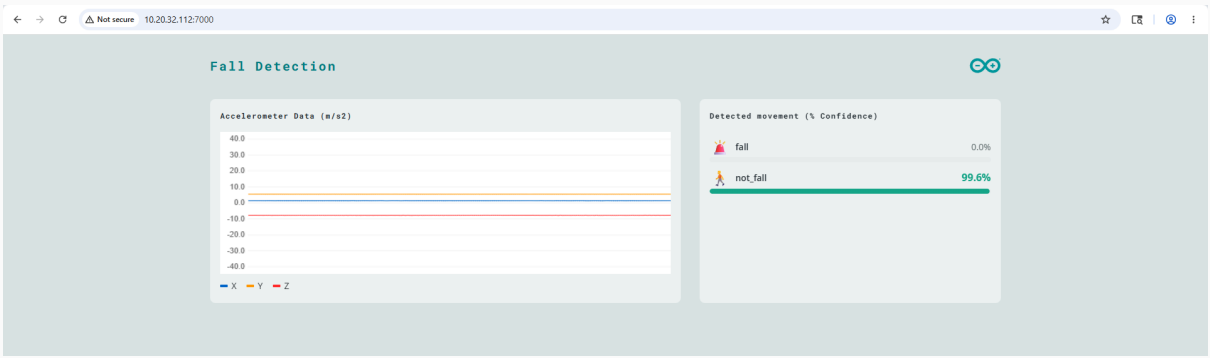
Goal: run the on-board application and watch your model react to real motion in real time.

Step 2.1 — Launch the application

1. Ensure the UNO Q is powered and the accelerometer module is connected.
2. Start the Fall Detection application on the board by clicking the **RUN** button on the top right of Arduino App Lab. This might take some time, watch the bottom panel App Launch for progress



3. Open the app's web dashboard in a browser to see live class probabilities and the motion chart.(`http://<device-ip>:<port>`)



✓ **Expected:** *The dashboard connects and streams live accelerometer values; with the board still, not_fall is the dominant class.*

Step 2.2 — Validate with motion

1. Hold the board still or move it gently—confirm not_fall stays high.
2. Simulate a fall: provide a jerk to the board. Watch the fall probability spike.
3. Repeat the motion and note how the confidence changes.

✓ **Expected:** *Gentle motion → not_fall dominant. A sharp drop/impact → fall probability rises above the confidence threshold.*

Step 2.3 — Observe and discuss

What to notice

- A harder impact gives higher fall confidence; a gentle drop gives lower confidence. Why? A softer fall has a weaker motion signature—and these gentle falls are exactly the hard case for real detectors.
- Recall the test results: the model caught about 95% of falls, missing a few — almost certainly the gentlest ones. You are watching that same limitation live.
- Try motions that might fool it—a quick sit, a hand clap, waving. Do any trigger a false 'fall'? These are the wrist-sensor ambiguities the dataset was designed to include.

Connecting back to the big idea

This is exactly why a real product pairs the model with a decision layer. The model is the perception (is this motion fall-like?); a small state machine would take a detected fall, wait, check for continued stillness, and confirm before raising an alarm—catching the gentle falls the model is unsure about while suppressing false alarms from clapping or sitting. Model plus logic, working together.